SOEN 387 — Assignment 1

The Waitlist

Contract, idempotency, errors, and evolution · Chapters 2–3 · ~2 weeks · individual

This is the first of three assignments that grow the **Encore** campus event-ticketing service one capability at a time. You are given a working Encore service (a small Flask app with an OpenAPI contract and a conformance suite). Your job is to **add a new capability** — and to add it the way the course teaches: contract first, semantics stated, errors that a program can act on, and changes that don't break existing callers.

The feature: a waitlist

When an event is **sold out**, a person can join a **waitlist**. The waitlist is ordered (first come, first served). A person can check their position and leave the list. (Automatically promoting the next person when a seat frees up is *not* required in this assignment — that logic arrives in Assignment 2, when seats gain a lifecycle. Here, focus on the boundary: contract, idempotency, errors, evolution.)

What you will build

1 · Contract first (OpenAPI)

Before writing any handler, extend encore-openapi.yaml with the new resources and operations. At minimum:

```
POST   /events/{eventId}/waitlist            join the waitlist
GET    /events/{eventId}/waitlist            list entries (ordered)
GET    /events/{eventId}/waitlist/{entryId}  one entry: your position
DELETE /events/{eventId}/waitlist/{entryId}  leave the waitlist
```

- Model a **waitlist entry** as a resource with a stable, opaque id (e.g. wl_5f3a). Give it a documented schema: id, eventId, position, createdAt.
- Give POST a request body identifying the person (e.g. a userId string — real auth arrives in Assignment 3). Keep the request schema strict; keep the response schema evolvable.
- Document each operation's success and error responses **by status code**, with response schemas referenced from components.

2 · Idempotency (Chapter 3)

- Joining the waitlist must be **idempotent**: a POST carrying an Idempotency-Key header, repeated with the same key and body, returns the **stored outcome** of the first attempt — the SAME entry, not a second one.
- Reusing a key with a *different* body is a client error, rejected loudly (a coded 422).
- State the retention window for keys in the contract description (≥ 24h). An unstated expiry cannot be relied upon.

3 · Error model (Chapter 3)

Define a small, documented catalog of stable, machine-readable error codes, each with its retryability. At least:

error code	status	meaning / when
event_not_found	404	no event has this id
event_not_full	409	the event still has seats — join by reserving, not by waitlisting (not retryable)
already_on_waitlist	409	this user already holds an entry for this event (not retryable)
idempotency_key_required	422	the POST carried no Idempotency-Key
idempotency_key_reuse	422	same key, different body (a client bug)
entry_not_found	404	no such waitlist entry

The error body carries the **code**, never prose as the only signal. Document that consumers must treat an *unrecognized* code as non-retryable — so the catalog can grow without breaking old clients.

4 · Evolution (Chapter 3)

- Adding the waitlist must be a **backward-compatible** change: every existing conformance test (events, reservations) must still pass unchanged. Old clients that know nothing of waitlists keep working.
- In your design note, classify each change you made as additive or breaking, and state the response-strictness policy you adopted (recall additionalProperties on responses).

5 · Conformance suite

- Extend the provided conformance suite with checks for the new operations: join returns a valid entry; a same-key retry replays the same entry; joining a *non-full* event yields event_not_full; an unknown entry yields entry_not_found.
- Your suite reads the contract and hits the running server — it must not import your handler code.

Deliverables

1. **encore-openapi.yaml** — the extended contract.
2. **The implementation** (Flask, extending the given app) that conforms to it.
3. **The extended conformance suite**, and a screenshot / capture of it passing (all old + new checks).
4. **A design note (1–2 pages)**: how you modeled the waitlist resource; where idempotency lives and why; your error catalog with retryability; and your compatibility argument (which changes are additive, and your response-strictness policy).
5. **An AI-collaboration log** (see below).

Submit as a single zip to the Moodle assignment, plus a one-command run instruction (e.g. `python app.py`).

Grading (100 pts)

Dimension	Pts	What full marks look like
Contract & resource modeling	25	sensible resources, opaque ids, strict requests / evolvable responses, responses documented per status code
Idempotency	25	join is idempotent on the key; same-key replay returns the SAME entry; key-reuse-with-different-body rejected; retention stated
Error model	20	documented stable codes with retryability; codes not prose; unknown-code rule stated
Backward-compatible evolution	15	all original conformance tests pass unchanged; compatibility argument is correct and explicit
Conformance suite & design note	15	new checks are meaningful and code-free; the note defends the key decisions

Automatic cap: if any original conformance test fails, the assignment is capped — a backward-incompatible change is the one failure this assignment is about preventing.

Hints

- Do the contract *first*, literally: write the YAML, then write the code to satisfy it. Running the old conformance suite after every change tells you the moment you break something.
- The idempotency store is a dict from key → stored outcome, consulted before the work and written after it — exactly as in Chapter 3. A stored 409 replays as a 409.

- “Join only when full” is the rule that makes event_not_full meaningful — it forces you to read the domain state, not just accept the request.

Using AI

You may use AI assistants to research, to draft the contract, and to scaffold code. But you must **understand and be able to defend every decision** — particularly where idempotency lives and why each error code is or isn’t retryable. Keep a short **AI-collaboration log**: what you asked, what the assistant produced, and at least two places where it was wrong, incomplete, or overconfident and you corrected it. Plausible-looking AI code is often subtly wrong exactly on idempotency and error semantics — the parts this assignment grades.